

0.3 Project Hints

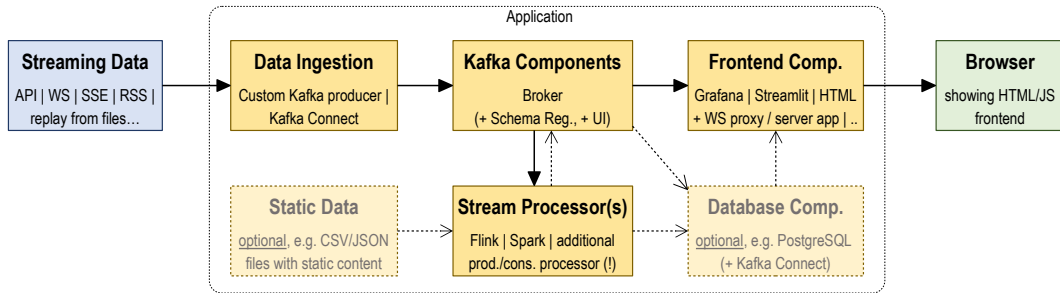
Real-Time Big Data Processing

Anton Dignös - anton.dignoes@unibz.it

Free University of Bozen-Bolzano

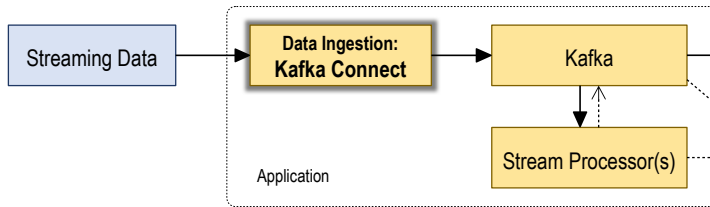
1. Reference Architecture

2. Implementation Hints



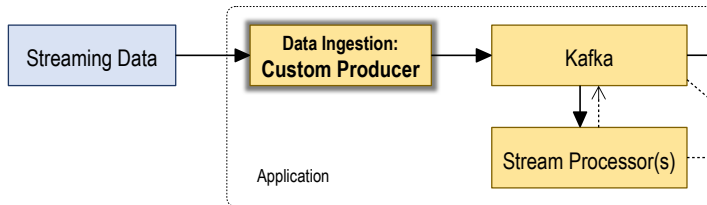
Project requirements map to this **reference architecture** (*variations are welcome!*):

- **data ingestion** components load one or more streams of data into **Kafka**
- **stream processors** read data from Kafka and transform it
→ may use **optional static data** and write results to Kafka and/or an **optional database**
- **frontend components** read data from Kafka or the optional database and create the Web UI



Deploy Kafka Connect **plugins** and configure **source connectors** to ingest data into Kafka

- suited to (replaying) **files**, **RDB**, **RSS** and **SSE** (streaming server events) sources
- suggestions
 - prefer **Json** over **Avro**, to avoid the need for **Confluent Schema Registry**
 - for **Json**, set `value.converter.schemas.enable = false` to write data in a cleaner format
 - for **files**, try to write data in **ascending timestamp** order to simplify downstream *watermarks*
 - use specific connector properties to **configure the topic** being written (e.g., retention)

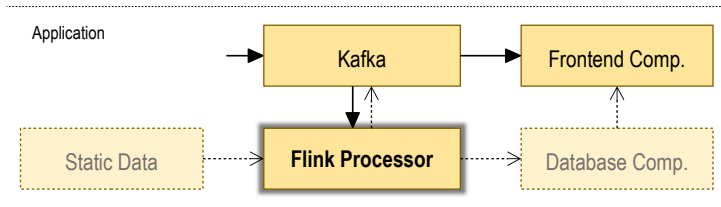


Custom application writing data from the source into Kafka via the **Producer API**

- suited to **any** data source, esp. polling **ReST APIs** & reading from **Web Sockets (WS)**
→ prefer this option if Kafka Connect is not (freely) available
- suggestions:
 - prefer **Json** to **Avro**, so to simplify your code and get rid of the **Confluent Schema Registry**
 - explicitly create & configure (e.g., retention) the Kafka topic using the Kafka **Admin API**
 - reuse **client library** for the data source & test writing to **stdout** instead of Kafka (`--dry-run`)
 - in Python, remember to call `poll()` after `produce()`, or writing may hang

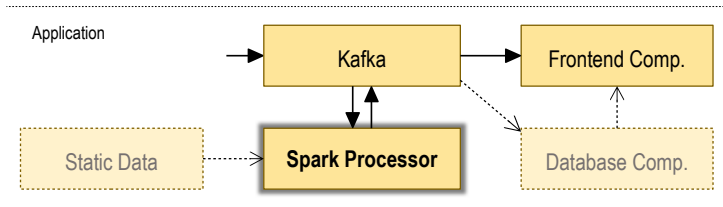
Data Ingestion – Examples

Strategy	Example	Notes
Custom Producer	lab_kafka_wiki (3) ↗ ↗	SSE data, in Python via <i>sseclient</i> ↗ , in Java via <i>okhttp-eventsourcing</i> ↗
	lab_flink_crypto (7) ↗	WS data, in Python via <i>websocket-client</i> ↗
	lab_flink_weather (6) ↗	API polling in Python via <i>requests</i> ↗ , identifying new data & also fetching historical data
Kafka Connect	lab_kafka_rss (4) ↗	RSS connector (Avro + Confluent Schema Registry)
	lab_spark_rss (9) ↗	RSS connector (Json)
	lab_kafka_opensky (5) ↗	CSV / API connectors + transformers & topic setup
	lab_spark_wiki (8) ↗	SSE connector + transformers & topic setup



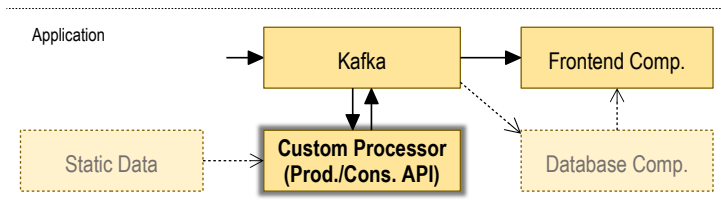
Transform/aggregate data with **DataStream/Table/SQL APIs**, including **complex event processing**

- suited to **complex aggregations** and **pattern matching (CEP)**
 - Flink has less limitations than Spark, e.g., when chaining aggregations
- suggestions
 - wait for input topic (or will fail) and setup output topic via **Kafka Admin API**
 - prefer **Table/SQL API**, using SQL queries to define data transformations where possible
 - pay attention to **defining watermarks** and possibly to configuring **source idleness** for Kafka
 - if there is a **database** (e.g., to drive Grafana), prefer writing directly to it



Transform/aggregate data with the **DataFrame/SQL API**, with support for **machine learning & NLP**

- suited to **simple aggregations** and **simple machine learning** tasks
 - for *one-record-at-a-time* ML/NLP may also use any Python/Java library called via Spark UDFs
- suggestions
 - wait for input topic availability and setup output topic via **Kafka Admin API**
 - pay attention to **defining watermarks** and to **aggregation chains limitations**
 - if there is a **database** (e.g., for Grafana), prefer writing to Kafka and then use Kafka Connect
 - beware of complex Spark installation on **Windows** (need for Hadoop *winutils* binaries)



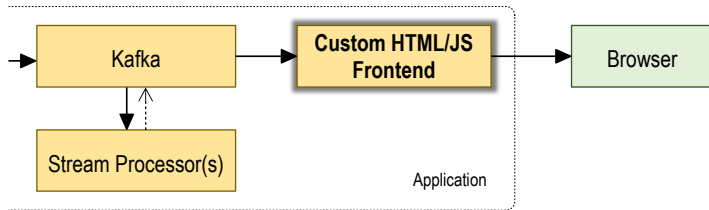
Perform **additional accessory processing** via custom code using the Kafka **Producer/Consumer API**

- suited to accessory tasks *complementing* Flink or Spark
 - e.g., dump some Kafka data, train a **machine learning model**, and store it to Kafka/disk
- suggestions
 - wait for input topic availability and setup output topic via **Kafka Admin API**
 - if there is a **database** (e.g., for Grafana), prefer writing to Kafka and then use Kafka Connect

Use of Spark / Flink is still required

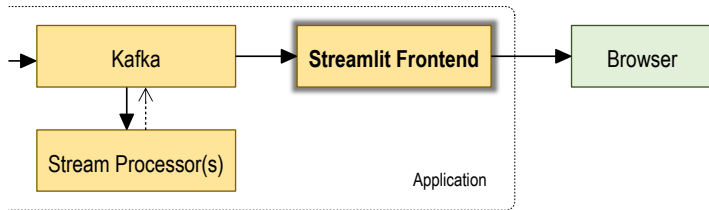
Processing – Examples

Strategy	Example	Notes
Custom	lab_kafka_wiki (3)  	Kafka in/out, simple rolling aggregation
	lab_kafka_rss (4)  	Kafka in/out, simple 1:1 message transformation using Avro and Schema Registry
Flink	lab_flink_weather (6)  	Kafka in/out, DataStream API window aggregations
	lab_flink_crypto (7)  	Kafka in, PostgreSQL out, window aggregation & stream join with Table/SQL API
Spark	lab_spark_wiki (8)  	Kafka in/out, rolling/window aggregation, stream join
	lab_spark_rss (9) 	Kafka in/out, load & evaluate Spark MLlib classifier



Arbitrary reactive HTML/JavaScript UI for **maximum flexibility**

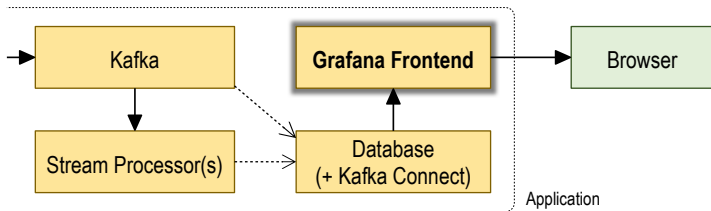
- may build on top of **JS libraries** such as *RxJS*, *ECharts* and *GridJS*
- option #1: **Kafka WebSocket Proxy** + web server (e.g., Apache)
 - remember setting a random *group ID* and try using a single *Kafka topic* (i.e., WS)
- option #2: **custom web app**
 - will serve both JSON content (e.g., `/data`) and a web page (e.g., `/index.html`) polling it
 - any language/framework, e.g., Python + Flask or Java + embedded web server



Define UI elements / tables / charts in **plain Python** based on **pandas** / **plotly**

- suited to **simple dashboards** only
→ limited customization capabilities (e.g., cannot even display links in tables)
- allows building a web UI with limited/no knowledge of HTML/JS
- reading from Kafka is tricky & needs a **background thread**: just copy & paste labs solution

Also checkout **Dash** [↗](#) (used in a project) which integrates with plotly and works like Streamlit



Store data in a **database** and configure one or more **complex dashboards** through **Grafana UI**

- need only knowing **SQL** to define charts via *queries* with *variables* set from user-supplied data
- more complex setup requiring a database such as **PostgreSQL**
 - the database can be populated either by the **stream processor** or via **Kafka Connect**
 - storing some **aggregate data** in the DB is fine, just do not dump the *whole* input stream!
- saving datasource & dashboard(s) is tricky: follow the advices given in the labs

Frontend – Examples

Solution	Example	Notes
Custom HTML/JS	lab_kafka_wiki (3) ↗	WS Proxy + RxJS + ECharts (bars) + Bootstrap
	lab_kafka_rss (4) ↗	Java app doing some text processing and serving an HTML page with a <i>jQuery</i> + <i>jQCloud</i> ↗ tag cloud
	lab_spark_wiki (8) ↗	WS Proxy + RxJS + ECharts (pie, bar, line) + Bootstrap
	lab_spark_rss (9) ↗	Python Flask app serving JSON data & an HTML page with an RxJS + GridJS ↗ table polling such data
Streamlit	lab_flink_weather (6) ↗	simple table view, no user controls
Grafana	lab_kafka_opensky (5) ↗	map/pie/bar/line charts with variables, PostgreSQL
	lab_flink_crypto (7) ↗	candlestick/bar charts with variables, PostgreSQL

How to Choose

Data Ingestion

- prefer **Kafka Connect** wherever it's usable, i.e., for RSS / SSE / RDB / file sources
- otherwise, go with a **custom producer** (Python is easier, but any language works here)

Stream Processor

- prefer **Flink** for complex aggregations, pattern matching (CEP), or if you are on Windows
- prefer **Spark** for machine learning, or if you have already familiarity with it
- for most applications, both are good so choose based on preferences

Frontend

- prefer **Streamlit** (or Dash) if you are familiar with Python, Pandas & co. but not Web tech.
- prefer **Grafana** if you are comfortable with SQL and prefer not to write code for the frontend
- prefer **custom HTML/JS** if you have web development skills and/or want to be creative
→ the **RxJS** reactive library works nicely here (instead, may easily do without RxPY/RxJava)

1. Reference Architecture

2. Implementation Hints

Development Environment

Software is affected by **operating system**, **CPU architecture**, version/distribution of **Java & Python**

- despite Java and use of Docker, issues may arise when native code is involved

Try to work in the same environment used to prepare the course material

- **Linux on Intel/AMD 64** (with some testing on an ARM 64 server)
 - beware of Mac OS X on ARM 64 (e.g., M1 CPUs) and native code (e.g., used by Spark NLP)
 - beware of Windows, with native code (e.g., Spark) and in general (e.g., line endings)
- **Java 11 OpenJDK** (Java 17 works too in most cases), developing in IntelliJ IDEA
- **Python 3.10 using pip** (vs. *conda*), developing in VSCode

If something does not work

- go back to the **examples** show in the course (tested in above environment) and try running them
 - if they run the problem is in the project code / configuration, otherwise report the issue
- in general, if **lab solutions** run locally, then the development environment is good

Using Docker

The use of **Docker** and **Docker Compose** is widespread (100% projects) & recommended

- inside Docker: *Kafka Broker/Connect/Schema Registry/UI/WS Proxy, Grafana, Postgres*
→ the alternative is to install, configure and run them natively (inconvenient!)
- outside Docker: *Flink/Spark* processors, custom producers, Java/Python *frontend code*
→ at least during development, can be *dockerized* later

Docker Compose recap

- `docker-compose.yml` – defines the Docker Compose application (see labs for examples)
- `.env` – sets configuration variables in `docker-compose.yml` (template: `.env.example`)
- `docker compose build` – generate custom Docker images (if needed, run to be sure)
- `docker compose up` – start the Docker Compose application
- `docker compose down` – stop the Docker compose application, keeping the data
- `docker compose down -v --remove-orphans` – stop everything, delete data & orphan containers
- `docker system prune` – discard unused Docker containers / images / volumes / networks / ...
- beware of network issues when host IP changes (e.g., on a laptop, restart the app)

Development Methodology

Develop **one component at a time**, then **iterate**

- start by configuring required Kafka components in `docker-compose.yml`
- implement the *data ingestion* logic and ensure data is stored in Kafka
- implement the core features of the *processor* (e.g., some window aggregations)
- implement the core features of the *frontend* to close the loop (e.g., some raw visualization)
- iterate to fix bugs and refine

Once a producer / processor is ready

- test it by **writing to stdout** (*dry-run*) to ensure it works correctly, then switch to writing to Kafka
→ if wrong data ends up in Kafka, use Kafka UI to clear or even drop the affected topic
- then, start the component and **let it run** while developing the rest
→ latent errors may pop up with time (e.g., some unexpected input to handle differently)

While developing, refer to provided **code examples**

- they are meant to show how to deal with common tasks and problems, use them!
- beside lecture slides, the labs **README** may provide further information (e.g., Grafana [🔗](#))